Component-oriented Design and Architecture

Overview

- Monolithic Virtual Store
  - Coupling
  - Dependency analysis
- Refactor to Components
- Component-based Virtual Store
  - Coupling
  - Dependency analysis
- Layered Architecture
- Summary

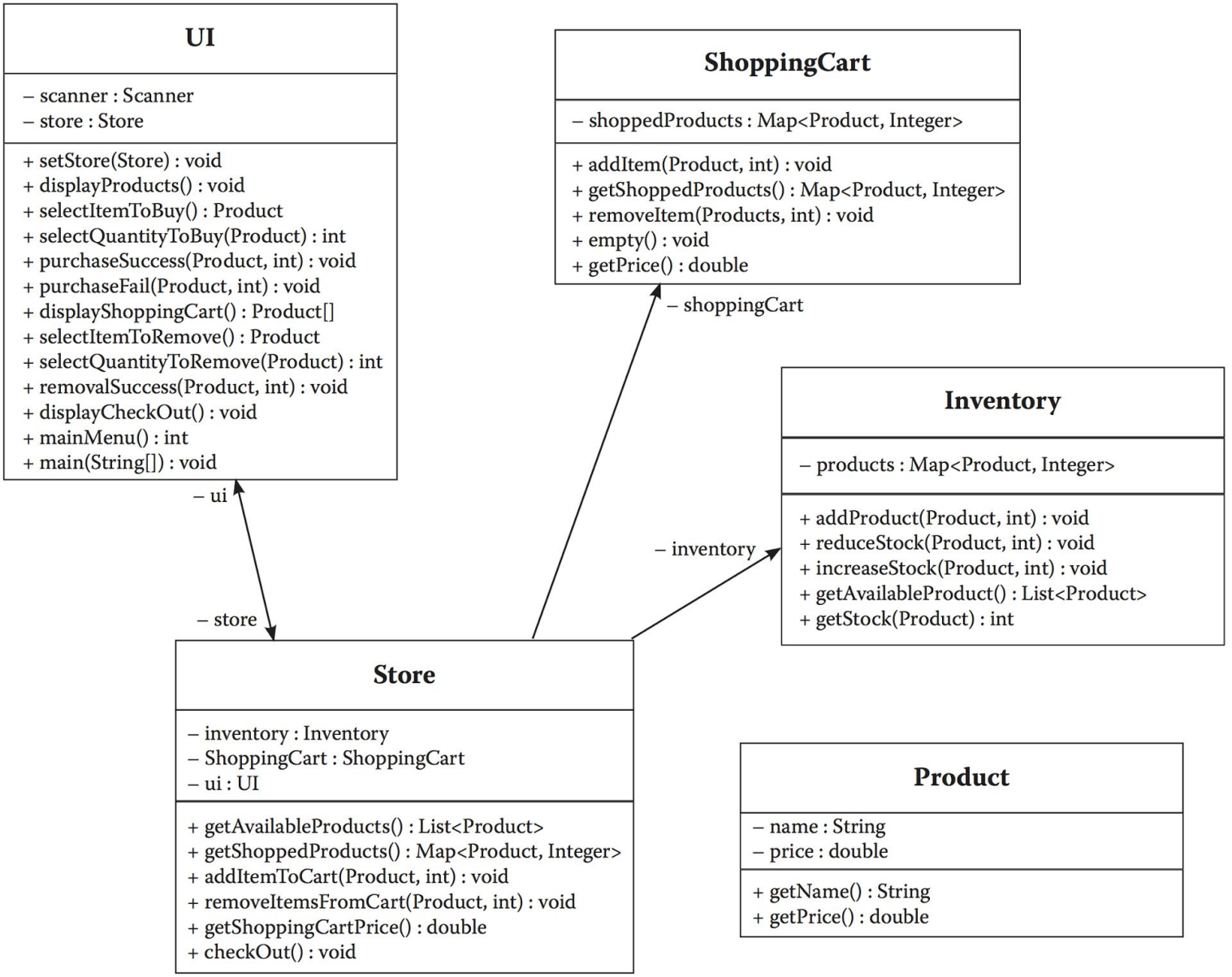
Componentization

Virtual Store

CASE/User story

- *Browse Products*—The user can browse through products available in the store and view an individual product's price description.
- *Shopping Cart*—The user can add multiple quantities of a product to the shopping cart. If sufficient stock is available, *Virtual Store* adds the selected items to the shopping cart. Items in the shopping cart can be removed or modified.
- *Check-out*—After adding all the required products into a shopping cart, the user can check out of the *Virtual Store*. Stock quantities of the purchased items are reduced in the *Virtual Store* inventory.

Monolithic Virtual Store



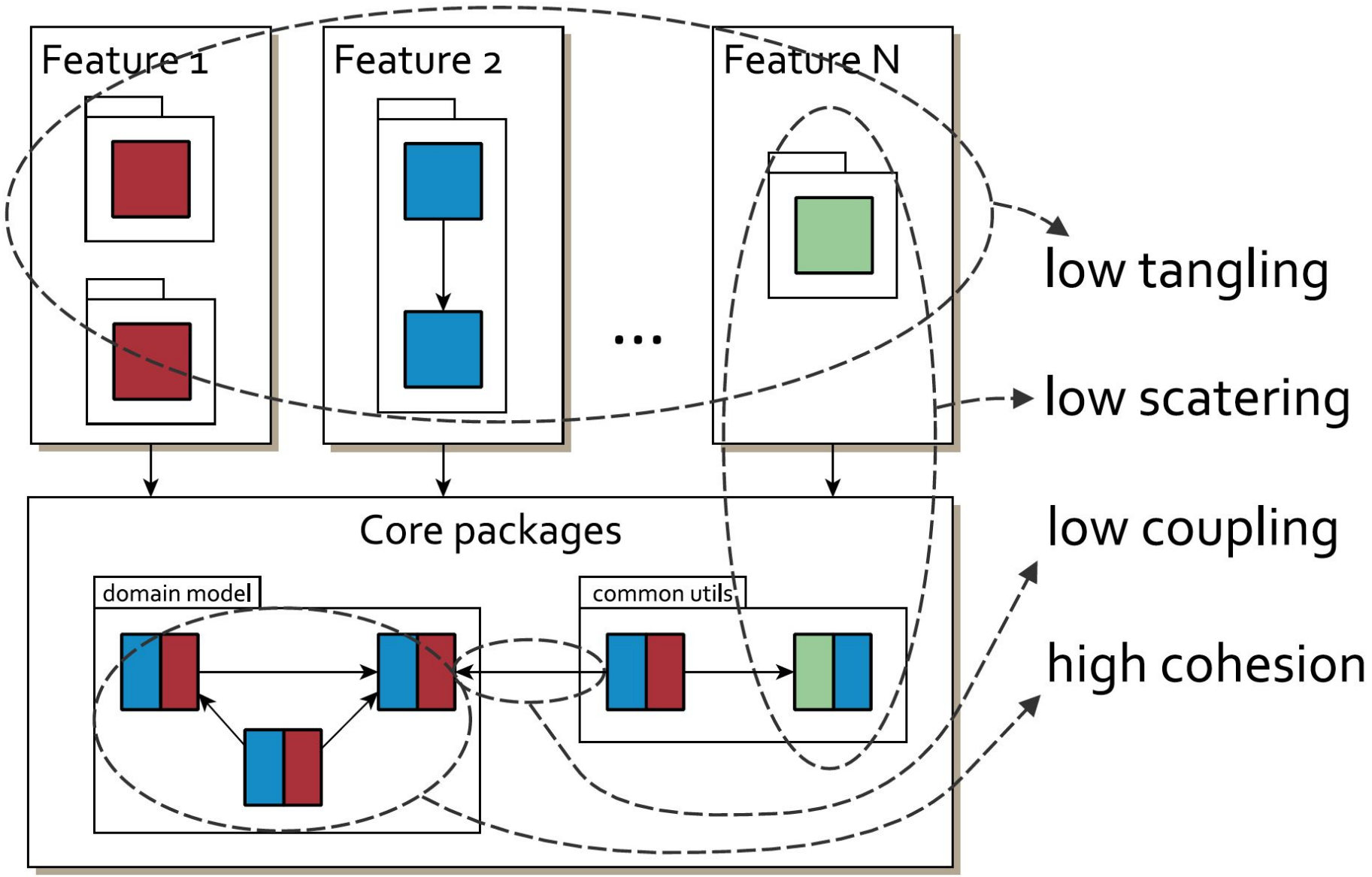
Coupling Metrics (Monolithic)

NUMBER	CLASS	DEPENDS ON	DEPENDENCY DEPTH
1.	Product	—	0
2.	Inventory	Product	1
3.	Shopping Cart	Product	1
4.	Store	Inventory, ShoppingCart, Product, UI	2
5.	UI	Store, Product	2

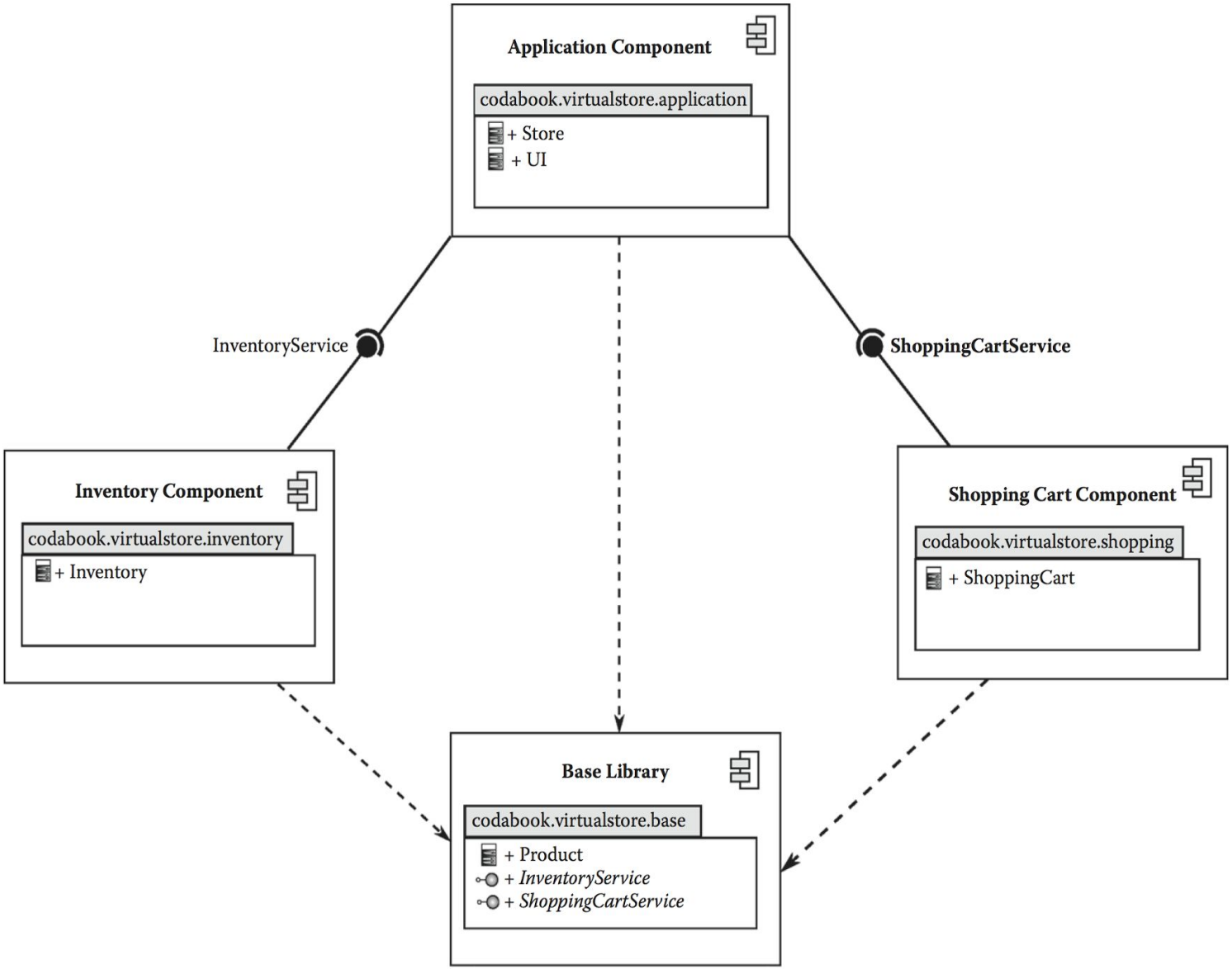
Componentization

1. Entity classes as a shared Common component (e.g. ***Product***)
2. Group business classes into separate components (e.g. **Inventory**)
3. Expose business functionality through provided interfaces (e.g. ***InventoryService***, ***ShoppingCartService***)
4. Group mutually dependent app. classes into Application components (e.g. ***UI***, ***Store***)
5. Include Service interfaces and entities in a base library

Feature-Oriented Design



Component-based Virtual Store



Measure Dependencies

Coupling Metrics (Components)

NUMBER	CLASS	DEPENDS ON	DEPENDENCY DEPTH
1.	`Product`	—	0
2.	`InventoryService`	`Product`	0
3.	`ShoppingCartService`	`Product`	0
4.	`Inventory`	`Product, InventoryService`	1
5.	`ShoppingCart`	`Product, ShoppingCartService`	1
6.	`Store`	`InventoryService, ShoppingCartService, Product, UI`	1
7.	`UI`	`Store, Product`	1

Build time Component dependencies

NUMBER	COMPONENT/LIBRARY	DEPENDS ON	DEPENDENCY DEPTH
1.	Base library	—	0
2.	Inventory component	Base library	1
3.	Shopping cart component	Base library	1
4.	Application component	Base library	1

Runtime Component Dependencies

NUMBER	COMPONENT/LIBRARY	DEPENDS ON	DEPENDENCY DEPTH
1.	Base library	—	0
2.	Inventory component	Base library	1
3.	Shopping cart component	Base library	1
4.	Application component	Base library, inventory component, shopping cart component	2

Accommodate Change

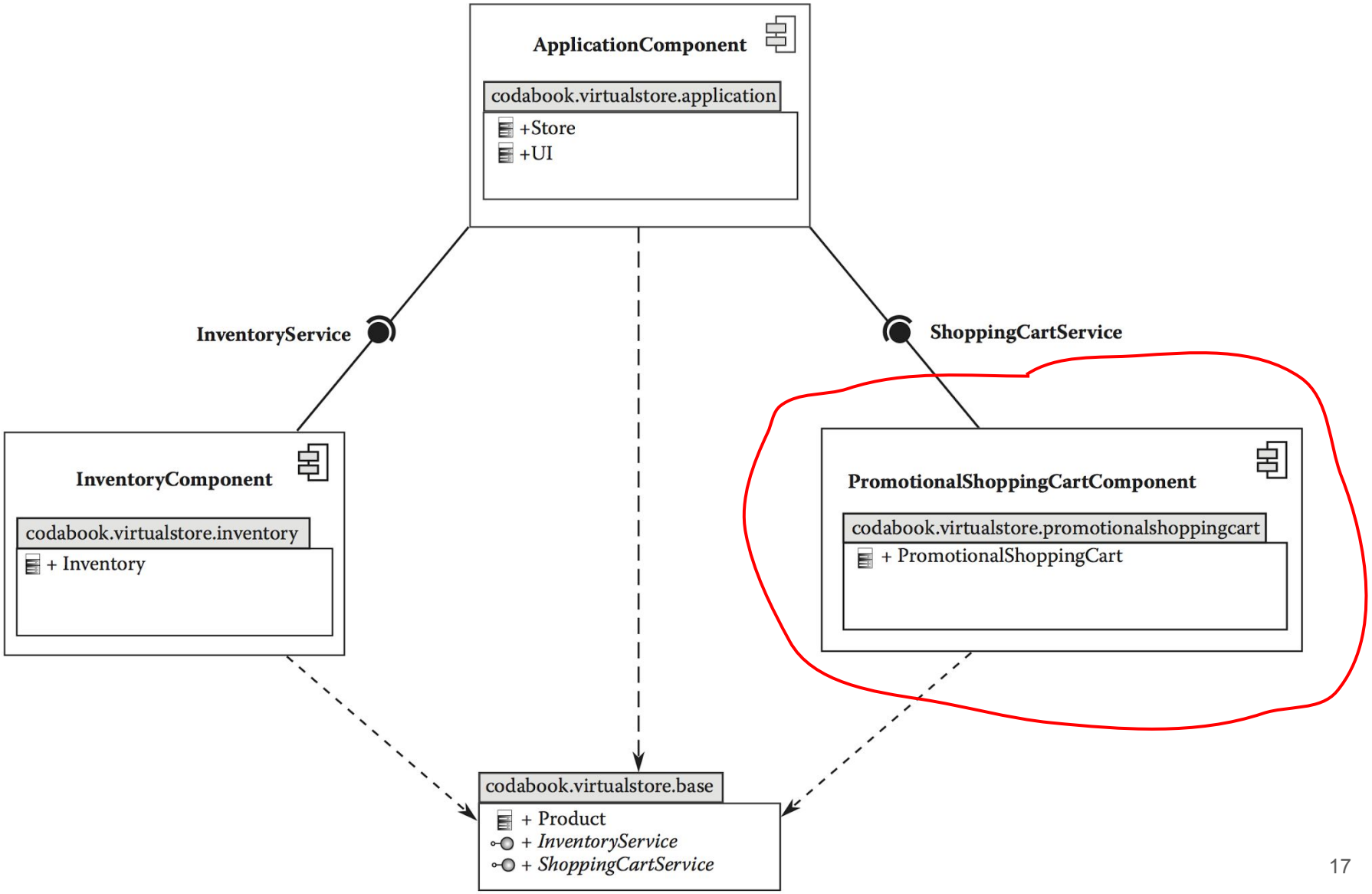
Shopping Cart Service

```
public interface ShoppingCartService {
        //… some code omitted for brevity
        public double getPrice();
}
```

Shopping Cart Impl

```
public class ShoppingCart implements ShoppingCartService,
   ComponentInterface {
          //… some code omitted for brevity
          public double getPrice() {
                 double totalPrice = 0;
                 for (Product product : shoppedProducts.keySet()) {
                        double unitPrice = product.getPrice();
                        int quantity = shoppedProducts.
                                                get(product);
                        double subTotal = unitPrice * quantity;
                        totalPrice + = subTotal;
                 }
                 return totalPrice;
          }
}
```

Promotional Shopping Cart



Promotional Shopping Cart impl

```
public class PromotionalShoppingCart implements
  ShoppingCartService, ComponentInterface {
        //… some code omitted for brevity
        public double getPrice() {
                double totalPrice = 0;
                for (Product product : shoppedProducts.keySet()) {
                        double unitPrice = product.getPrice();
                        int quantity = shoppedProducts.
                                                get(product);
                        double subTotal = unitPrice * quantity;
                        totalPrice + = subTotal;
                }
                return 0.9*totalPrice;
        }
}
```

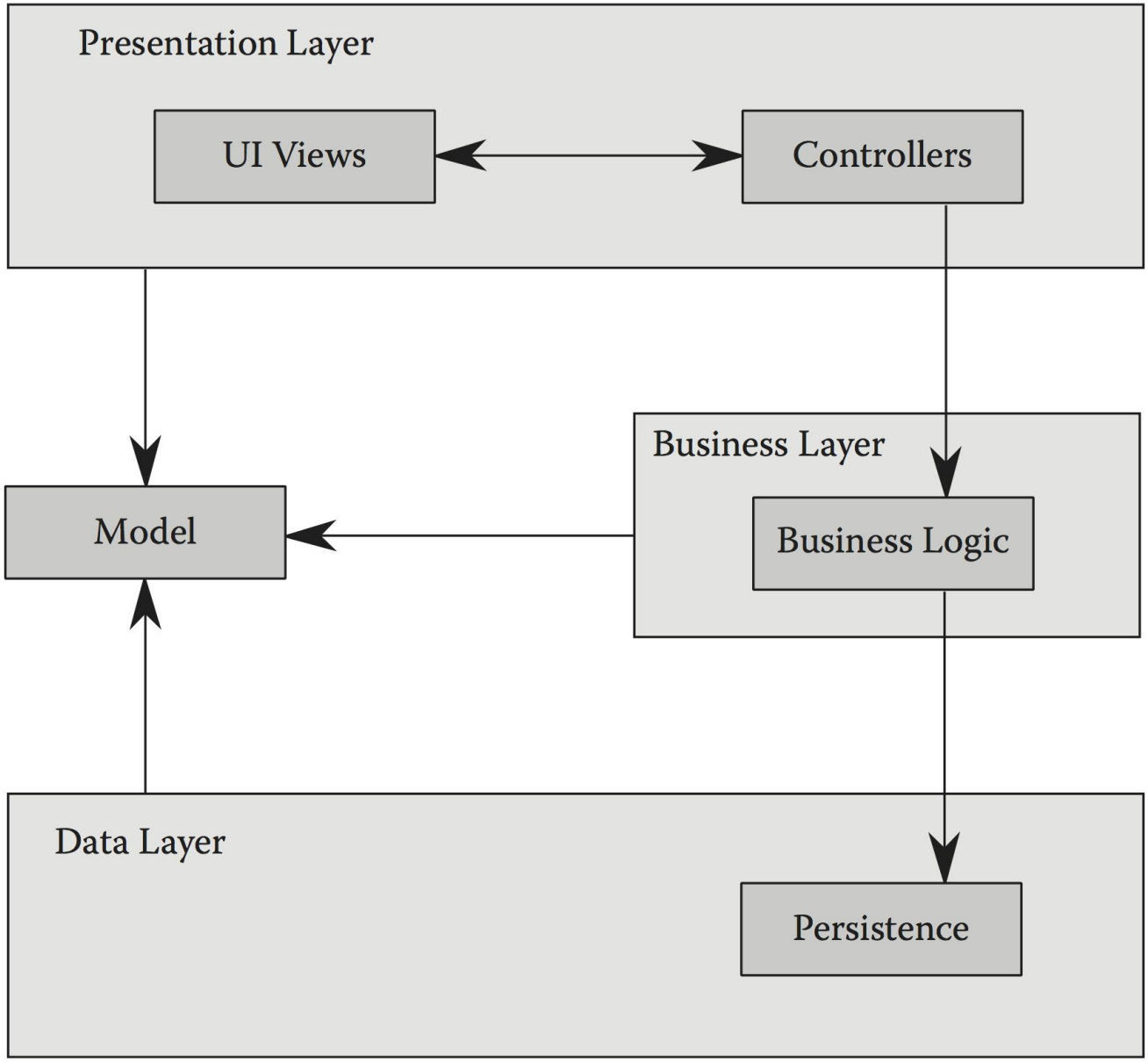
Layered Architecture

Point of Sale example

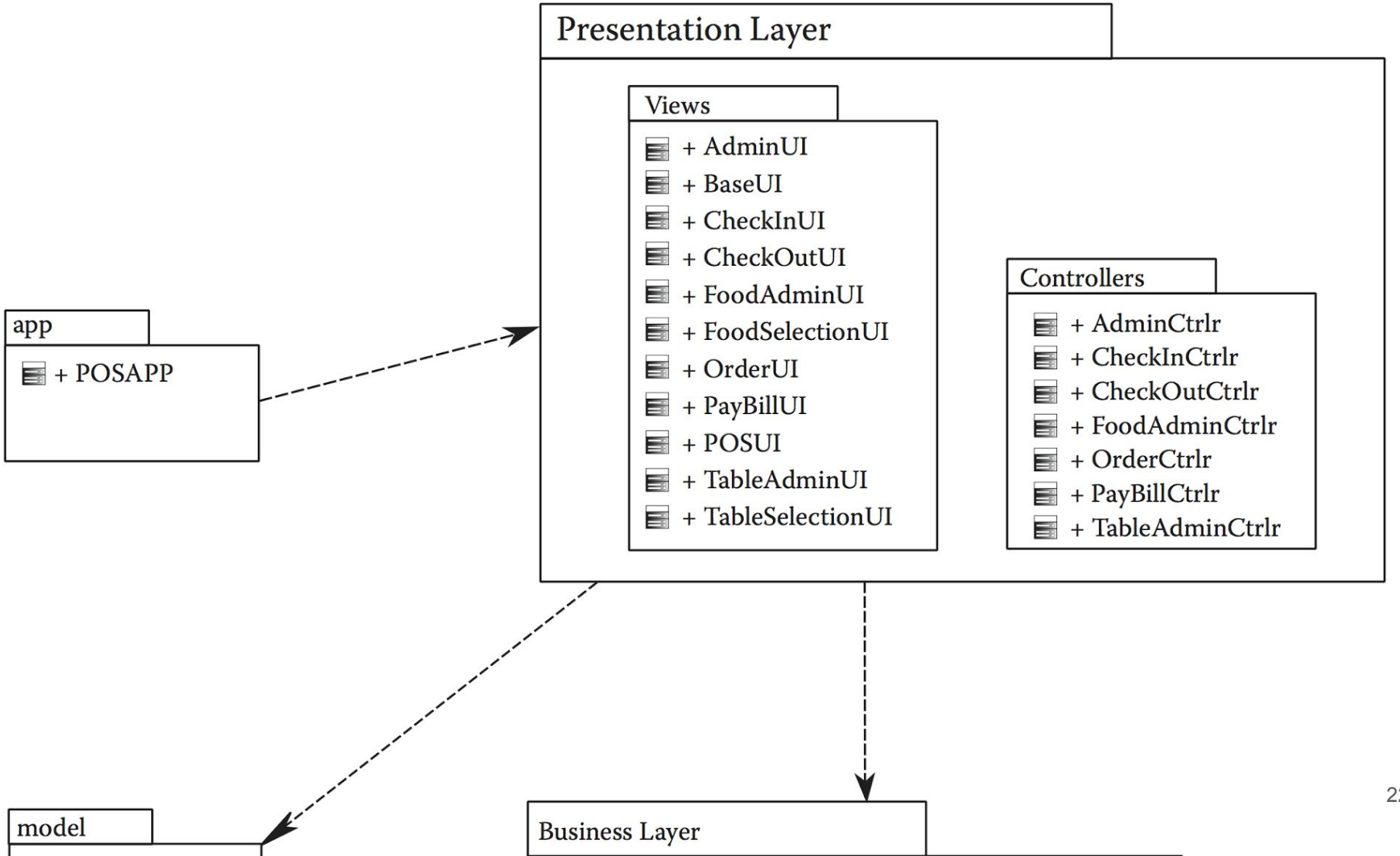
Restaurant Point of Sale (POS)

- *Guests Check-In*—This use case is invoked by the waiters when new guests arrive at the restaurant. POS displays a list of empty tables, and the waiter chooses an empty table and seats the guests at that table.
- *Place Order*—When guests from a table order food, the waiter invokes this use case. POS prompts the waiter for the table number, food item, and quantity ordered. POS consolidates and maintains orders against each table.
- *Modify Order*—The waiter can modify the quantity of any order already placed.
- *Cancel Order*—The waiter can cancel any order already placed.
- *Pay Bill*—The waiter invokes this use case to print the bill and collect payment for all items ordered from a table.
- *Guests Check-out*—When guests from a table leave, the waiter invokes this use case to mark the table as empty.
- *Collections Report*—At any point in time, the POS user can look at all the past payments collected.

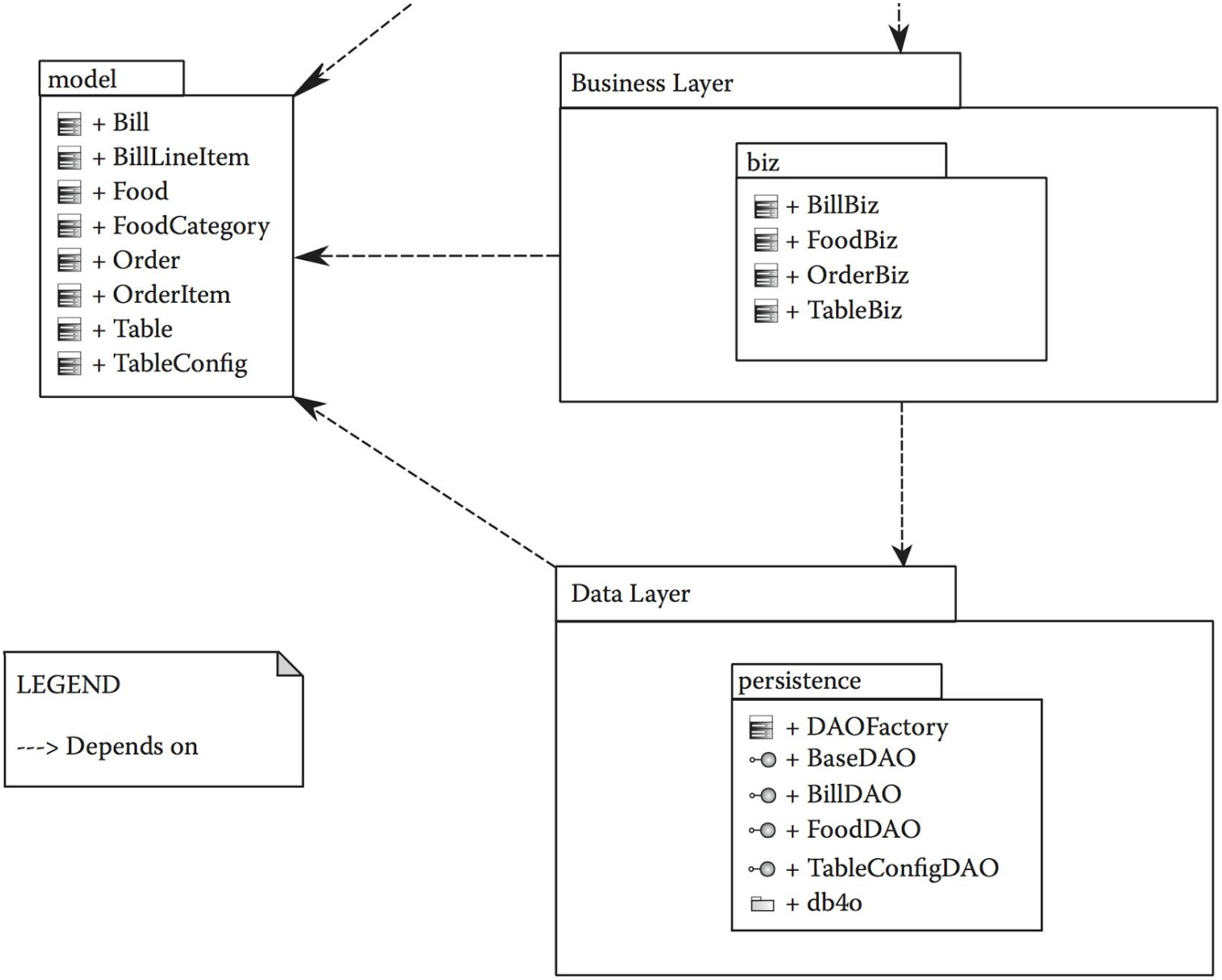
Classical Layered Architecture



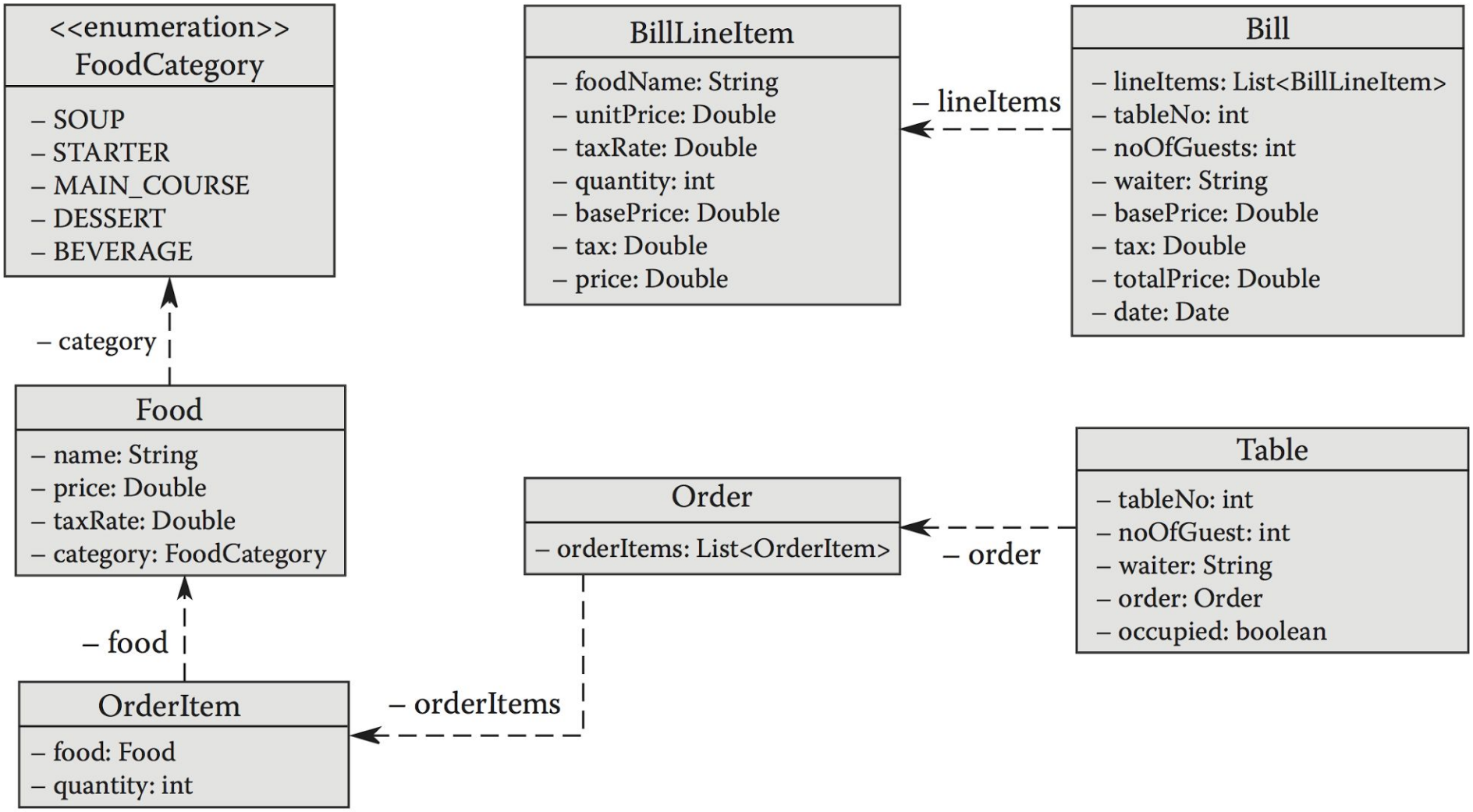
Point of sale



POS continued...



POS Data


<<enumeration>>
FoodCategory
– SOUP
– STARTER
– MAIN_COURSE
– DESSERT
– BEVERAGE
– category
Food
– name: String
– price: Double
– taxRate: Double
– category: FoodCategory
– food
OrderItem
– food: Food
– quantity: int
BillLineItem
– foodName: String
– unitPrice: Double
– taxRate: Double
– quantity: int
– basePrice: Double
– tax: Double
– price: Double
– lineItems
Bill
– lineItems: List<BillLineItem>
– tableNo: int
– noOfGuests: int
– waiter: String
– basePrice: Double
– tax: Double
– totalPrice: Double
– date: Date
Order
– orderItems: List<OrderItem>
– orderItems
– order
Table
– tableNo: int
– noOfGuest: int
– waiter: String
– order: Order
– occupied: boolean

Layered Component-based Architecture

Purpose

- Improve maintainability
- Extract reusable parts for storage and future reuse
- Example: POS is supposed to charge a gratuity (tip) of 15% for any guest group consisting of 8 or more members.

General Componentization Pattern

- Each **UI** component consists of a necessary view and **controller** objects
- Each **business** component consists of a necessary business object and a **DAO** for persistence.
- The objects from the **model** are packaged together as an object library (common), which is referred by each of these components.

Component-based Business

Bill Component

- Collect Payment for a Table
- Maintain Past Bills and Report

Order Component

- Manage Orders (add, modify, remove order items)

Table Component

- Maintain List of Tables
- Block and Release Tables

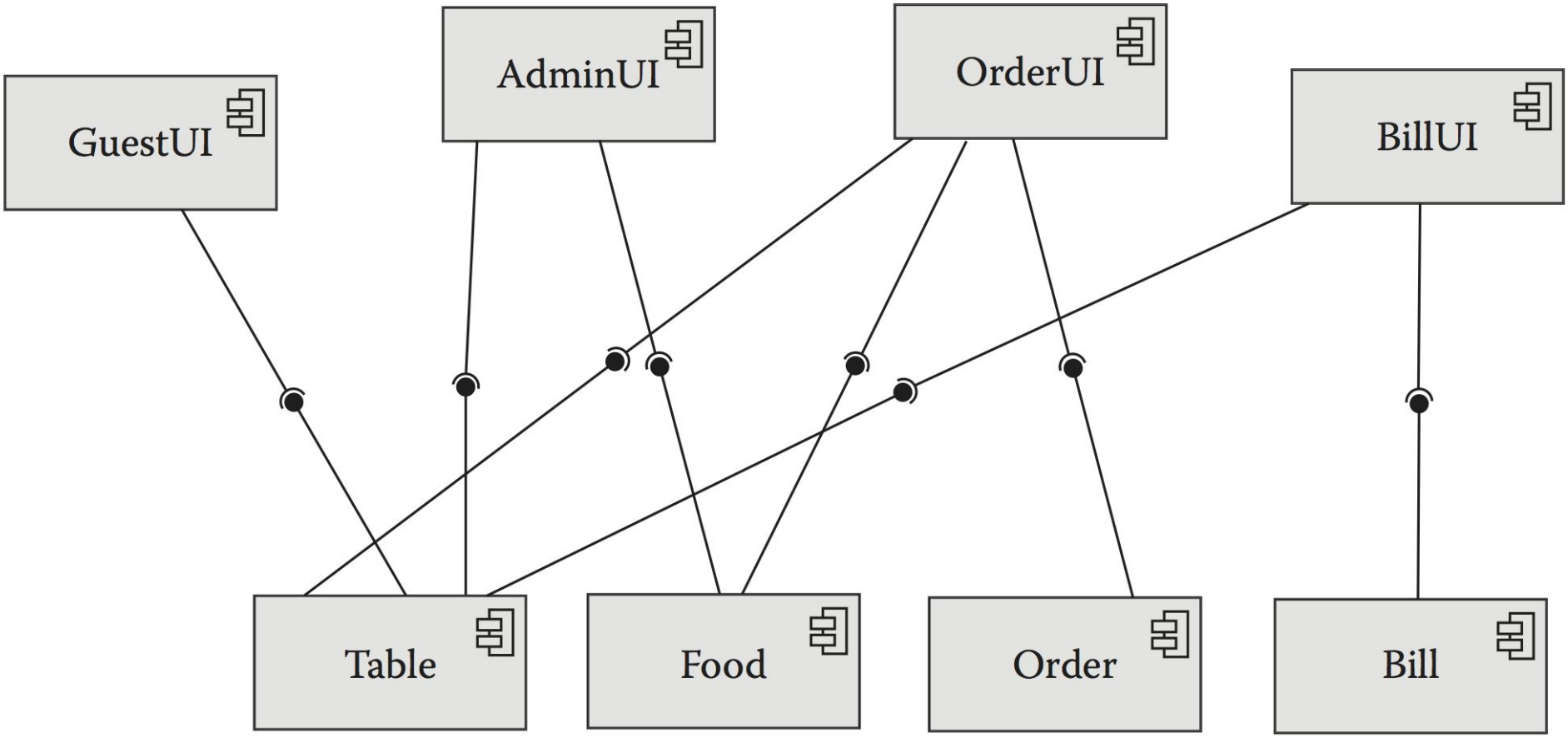
Food Component

- Add Food
- Modify Food
- Delete Food

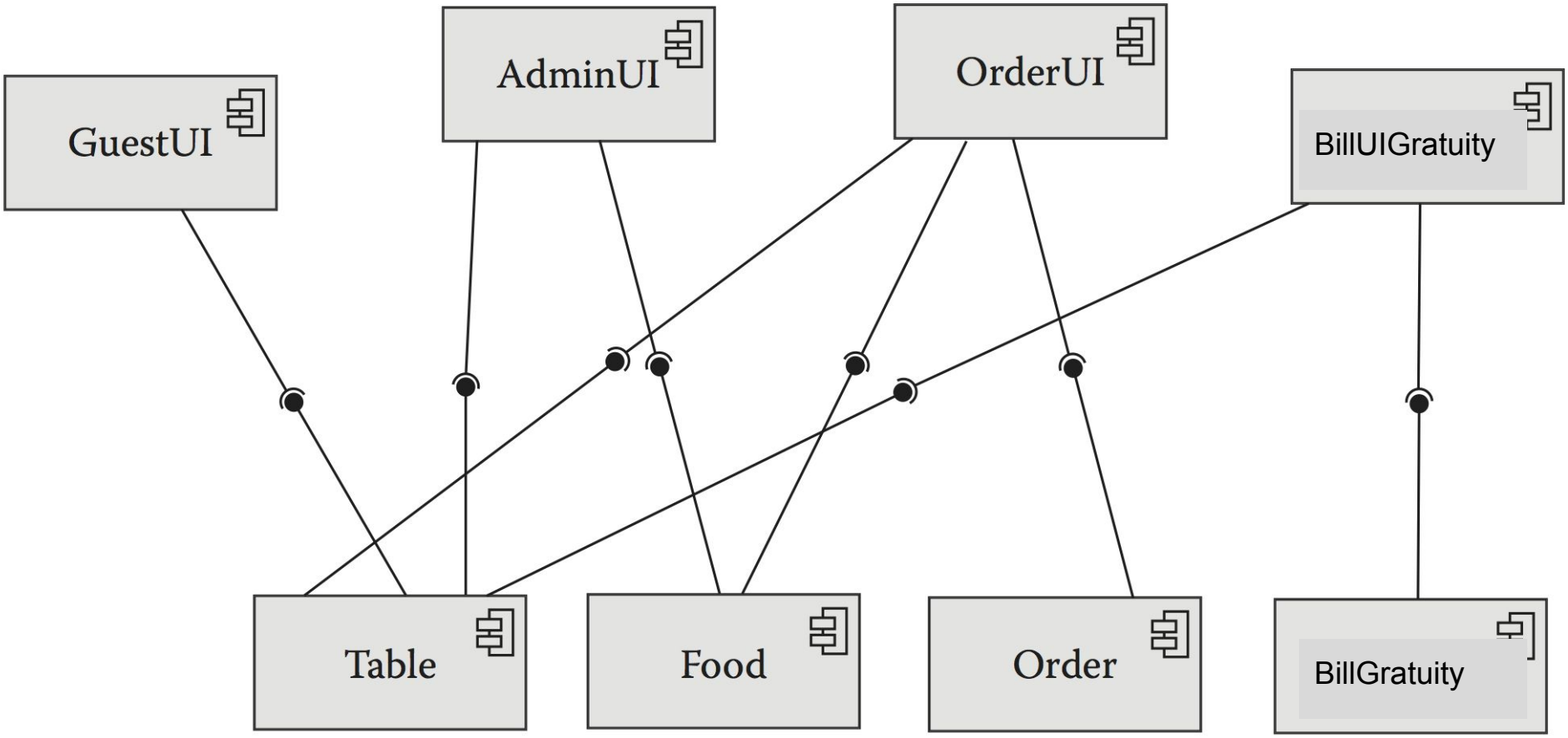
mapping Layers to Components

COMPONENT	OBJECTS	LAYER
GuestUI	CheckInUI, CheckOutUI, CheckInCtrlr, CheckOutCtrlr	Presentation
AdminUI	AdminUI, FoodAdminUI, TableAdminUI, FoodAdminCtrlr, TableAdminCtrlr	Presentation
OrderUI	OrderUI, OrderCtrlr	Presentation
BillUI	PayBillUI, PayBillCtrlr	Presentation
TableBiz	TableBiz, TableDAO	Business & Data
FoodBiz	FoodBiz, FoodDAO	Business & Data
OrderBiz	OrderBiz, OrderDAO	Business & Data
BillBiz	BillBiz, BillDAO	Business & Data

Component-based POS

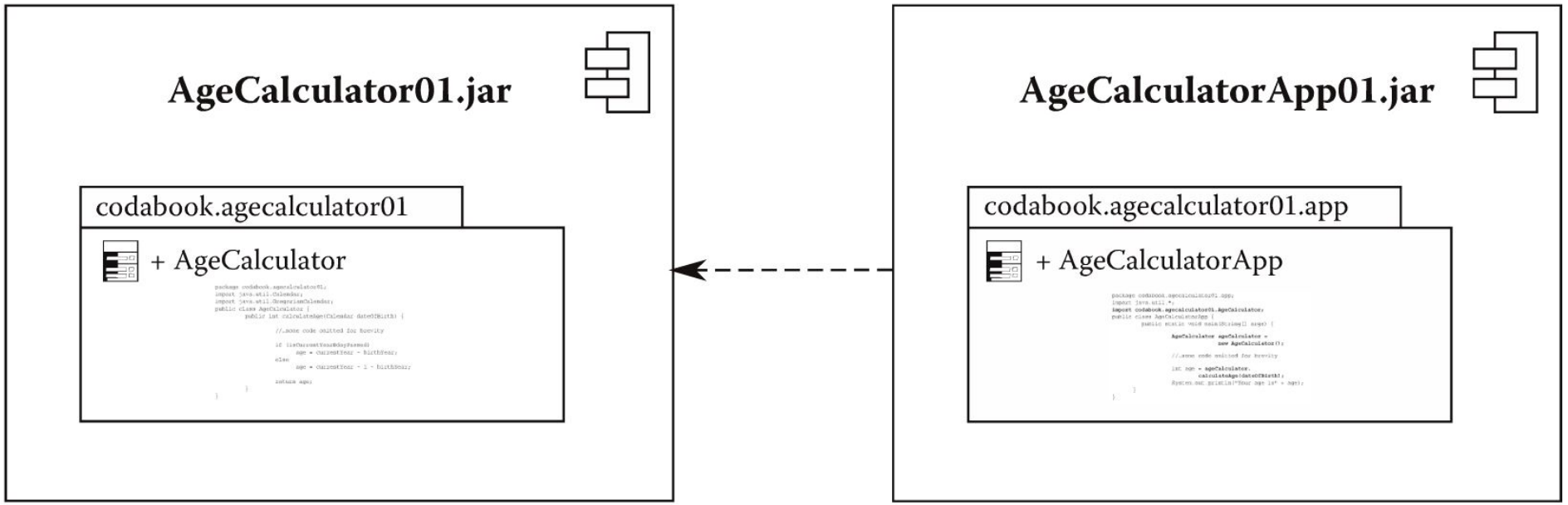


Component replacement

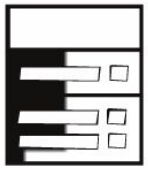


Java and Components

JARs and Packages



codabook.agecalculator01.app



+ AgeCalculatorApp

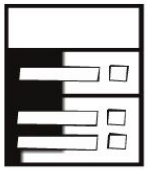
```
package codabook.agecalculator01.app;
import java.util.*;
import codabook.agecalculator01.AgeCalculator;
public class AgeCalculatorApp {
        public static void main(String[] args) {

                AgeCalculator ageCalculator =
                                new AgeCalculator();

                //…some code omitted for brevity

                int age = ageCalculator.
                        calculateAge(dateOfBirth);
                System.out.println("Your age is" + age);
        }
}
```

codabook.agecalculator01



+ AgeCalculator

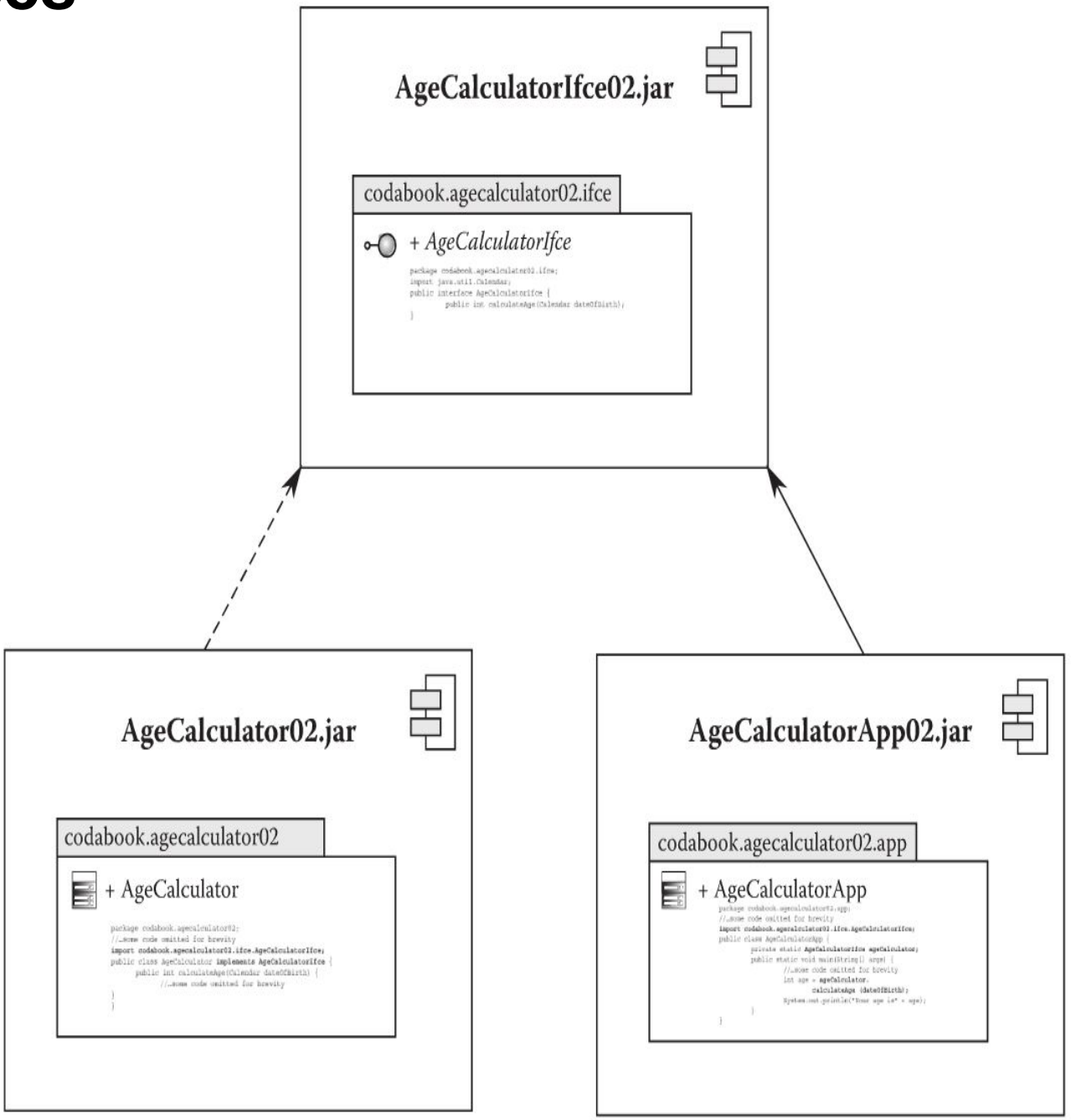
```
package codabook.agecalculator01;
import java.util.Calendar;
import java.util.GregorianCalendar;
public class AgeCalculator {
        public int calculateAge(Calendar dateOfBirth) {

                //…some code omitted for brevity

                if (isCurrentYearBdayPassed)
                      age = currentYear - birthYear;
                else
                      age = currentYear - 1 - birthYear;

                return age;
        }
}
```

Interfaces


AgeCalculatorIfce02.jar
codabook.agecalculator02.ifce
+ AgeCalculatorIfce
AgeCalculator02.jar
codabook.agecalculator02
+ AgeCalculator
AgeCalculatorApp02.jar
codabook.agecalculator02.app
+ AgeCalculatorApp

codabook.agecalculator02.ifce



+ *AgeCalculatorIfce*

```
package codabook.agecalculator02.ifce;
import java.util.Calendar;
public interface AgeCalculatorIfce {
         public int calculateAge(Calendar dateOfBirth);
}
```

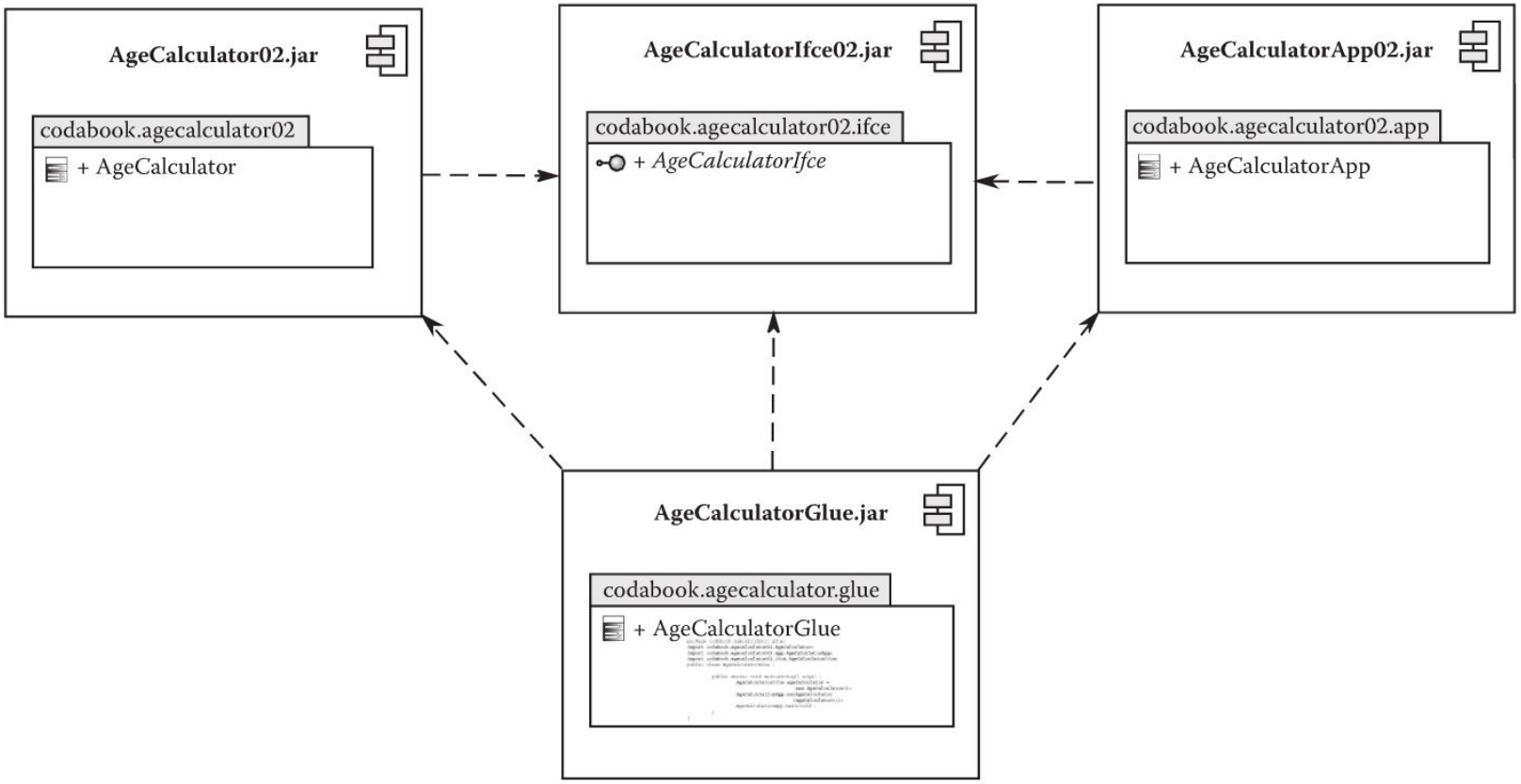
codabook.agecalculator02.app



+ AgeCalculatorApp

```
package codabook.agecalculator02.app;
//…some code omitted for brevity
import codabook.agecalculator02.ifce.AgeCalculatorIfce;
public class AgeCalculatorApp {
        private static AgeCalculatorIfce ageCalculator;
        public static void main(String[] args) {
                //…some code omitted for brevity
                int age = ageCalculator.
                        calculateAge (dateOfBirth);
                System.out.println("Your age is" + age);
        }
}
```

Assembly Clue



codabook.agecalculator.glue

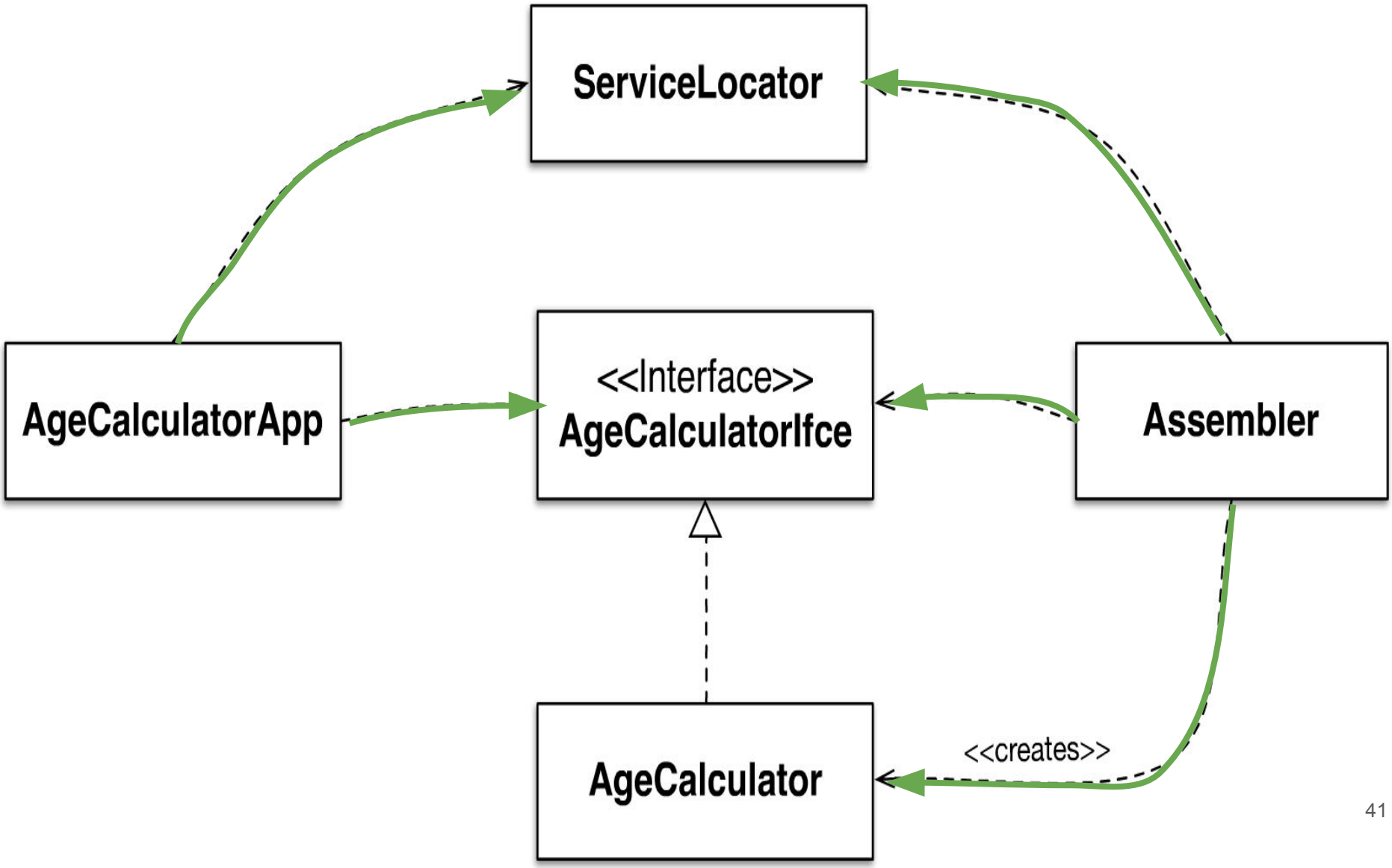


+ AgeCalculatorGlue

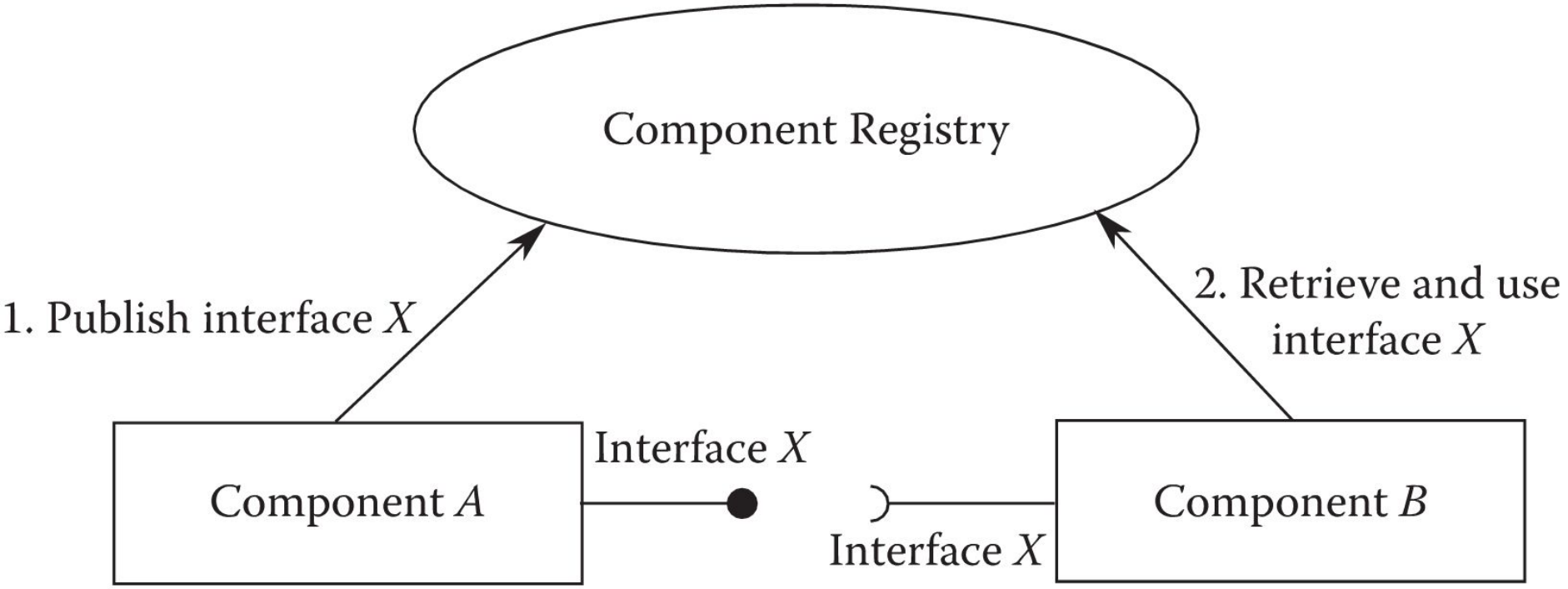
```
package codabook.agecalculator.glue;
import codabook.agecalculator02.AgeCalculator;
import codabook.agecalculator02.app.AgeCalculatorApp;
import codabook.agecalculator02.ifce.AgeCalculatorIfce;
public class AgeCalculatorGlue {

        public static void main(String[] args) {
                AgeCalculatorIfce ageCalculator =
                                        new AgeCalculator();
                AgeCalculationApp.setAgeCalculator
                                       (ageCalculator););
                AgeCalculationApp.main(null);
        }
}
```

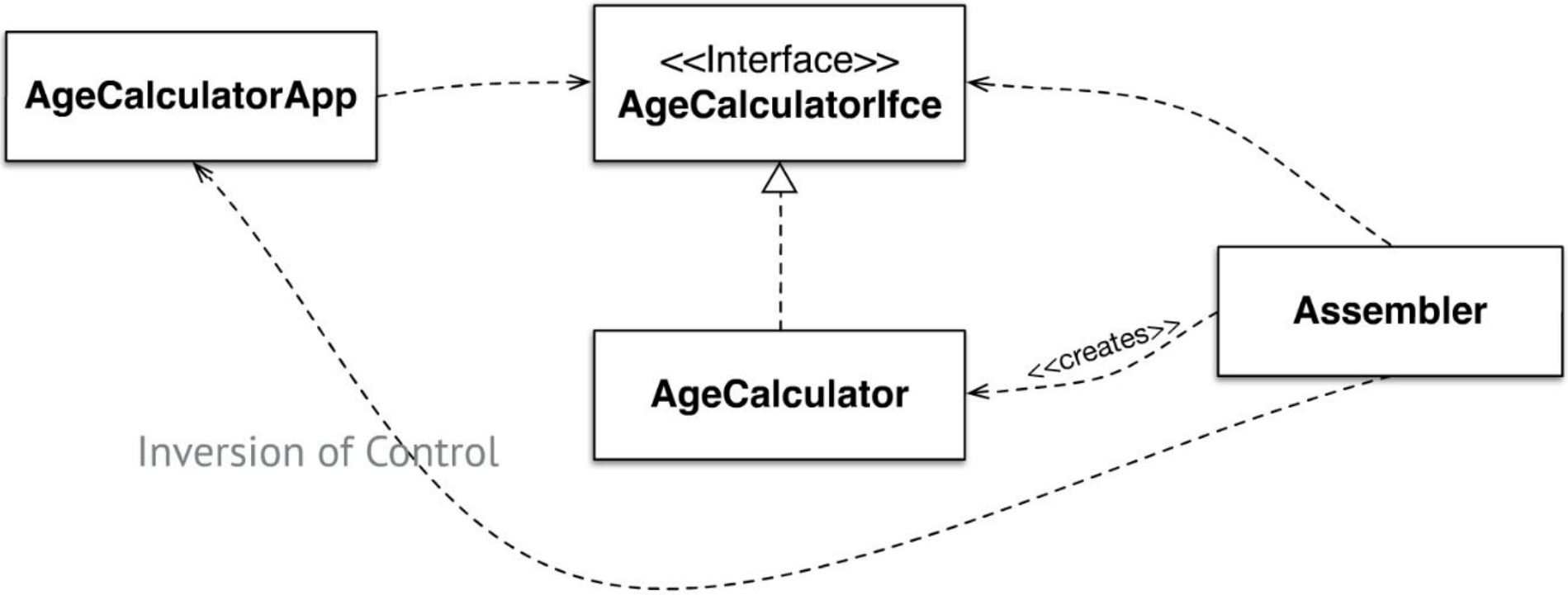
Service Locator

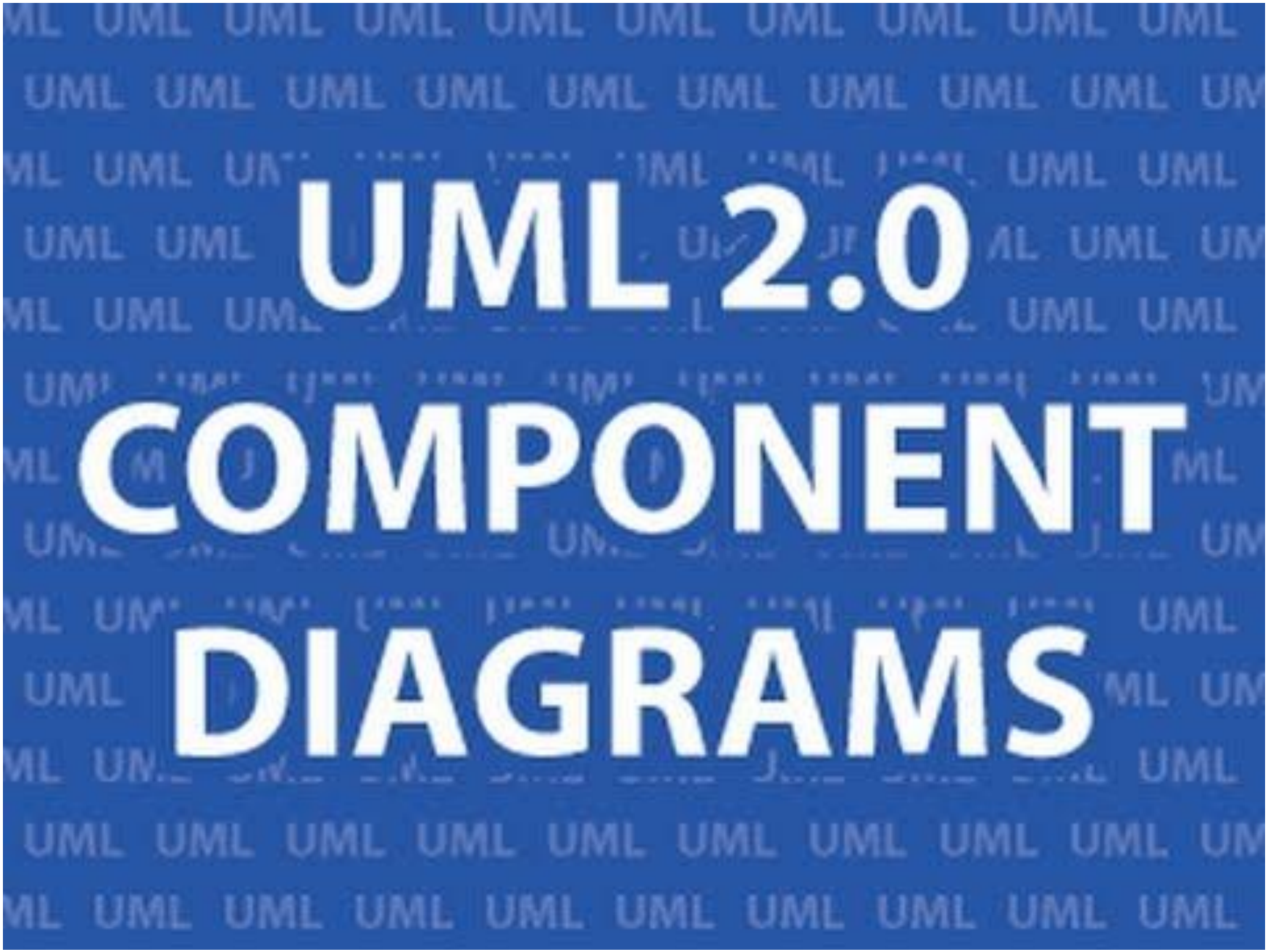


Whiteboard Component Model

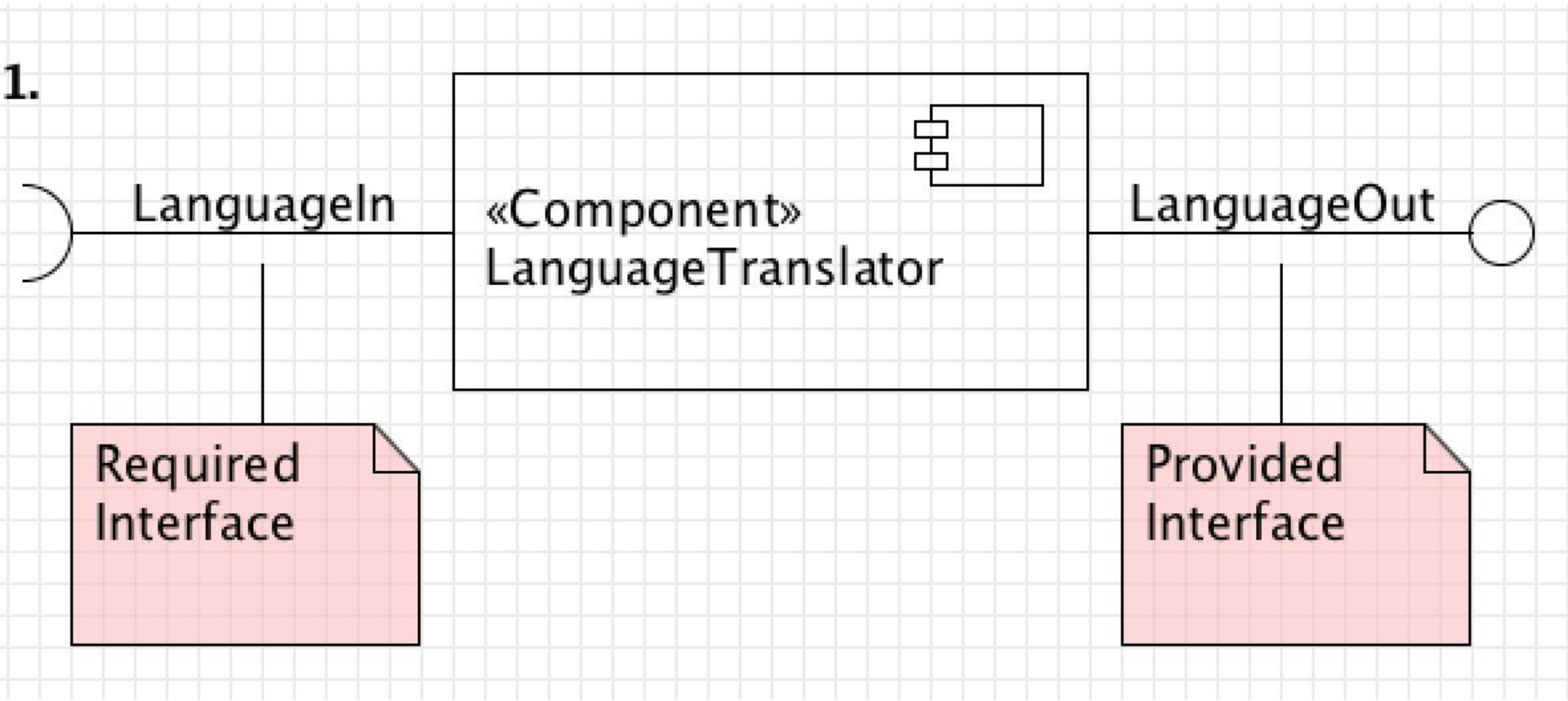


Dependency Injection

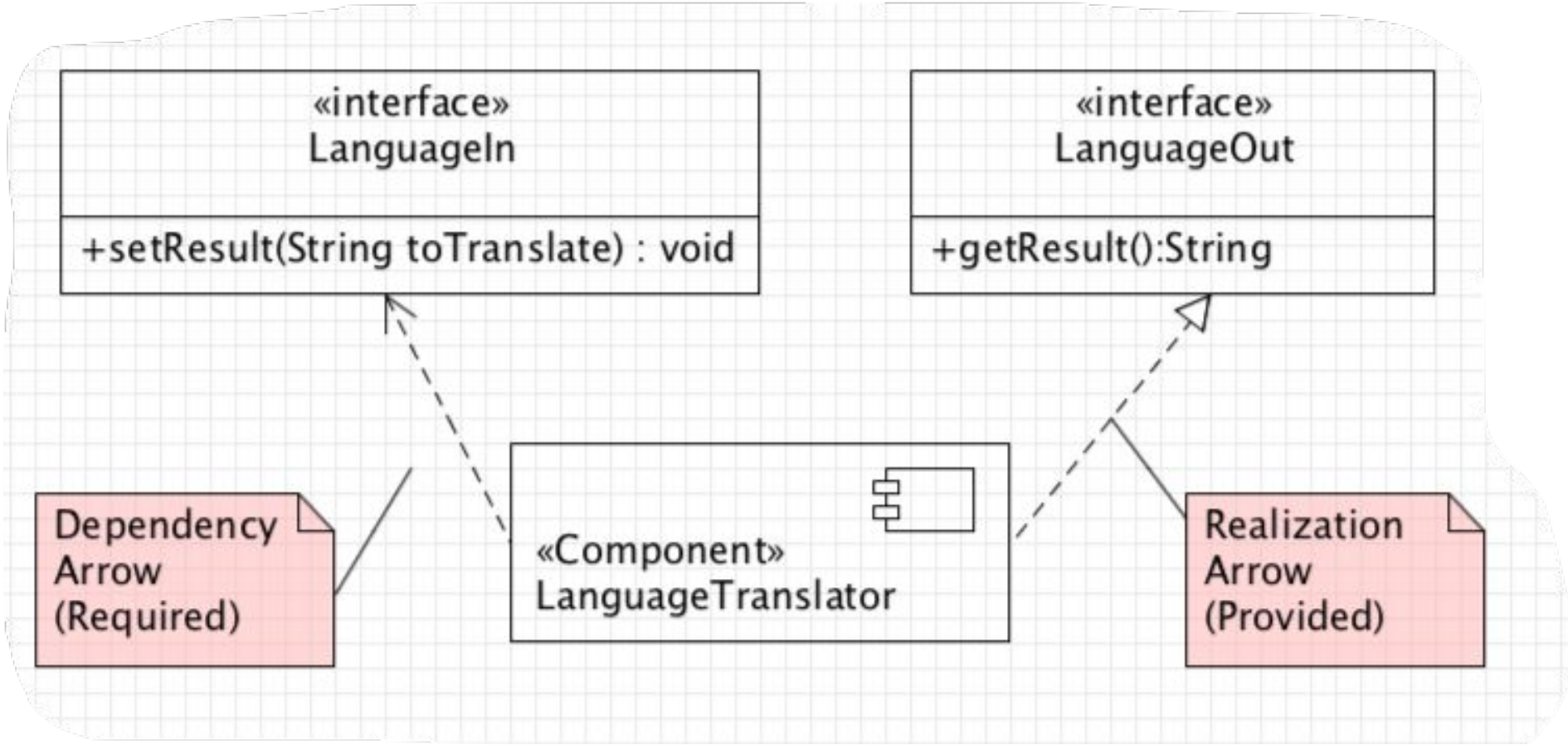



UML 2.0
COMPONENT
DIAGRAMS

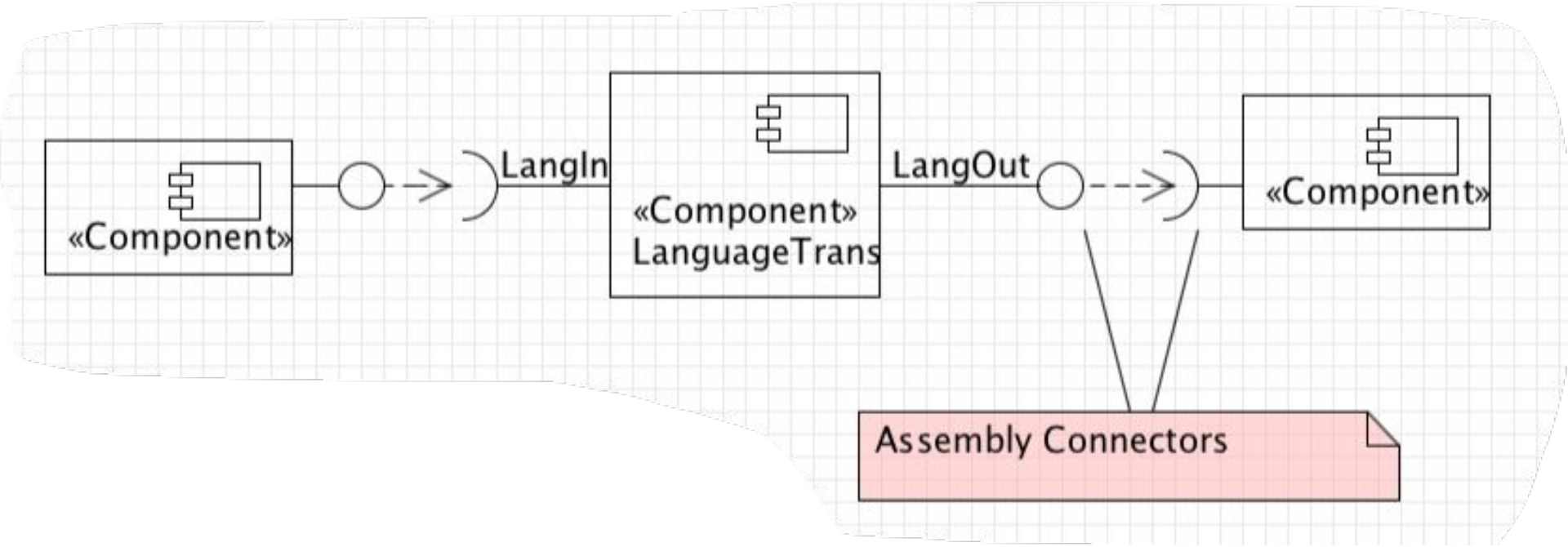
Required and Provided Interfaces



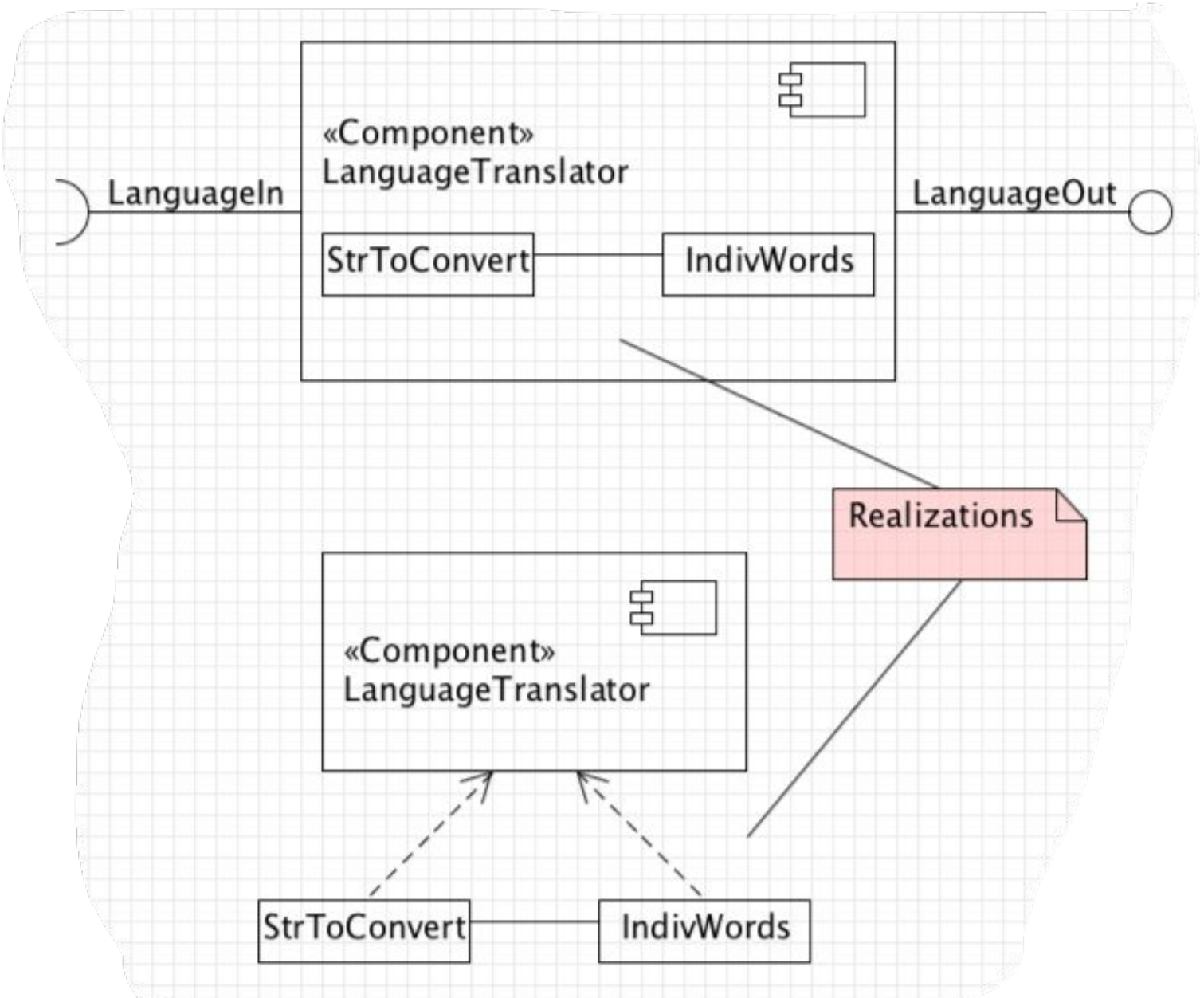
Required and Provided Interfaces



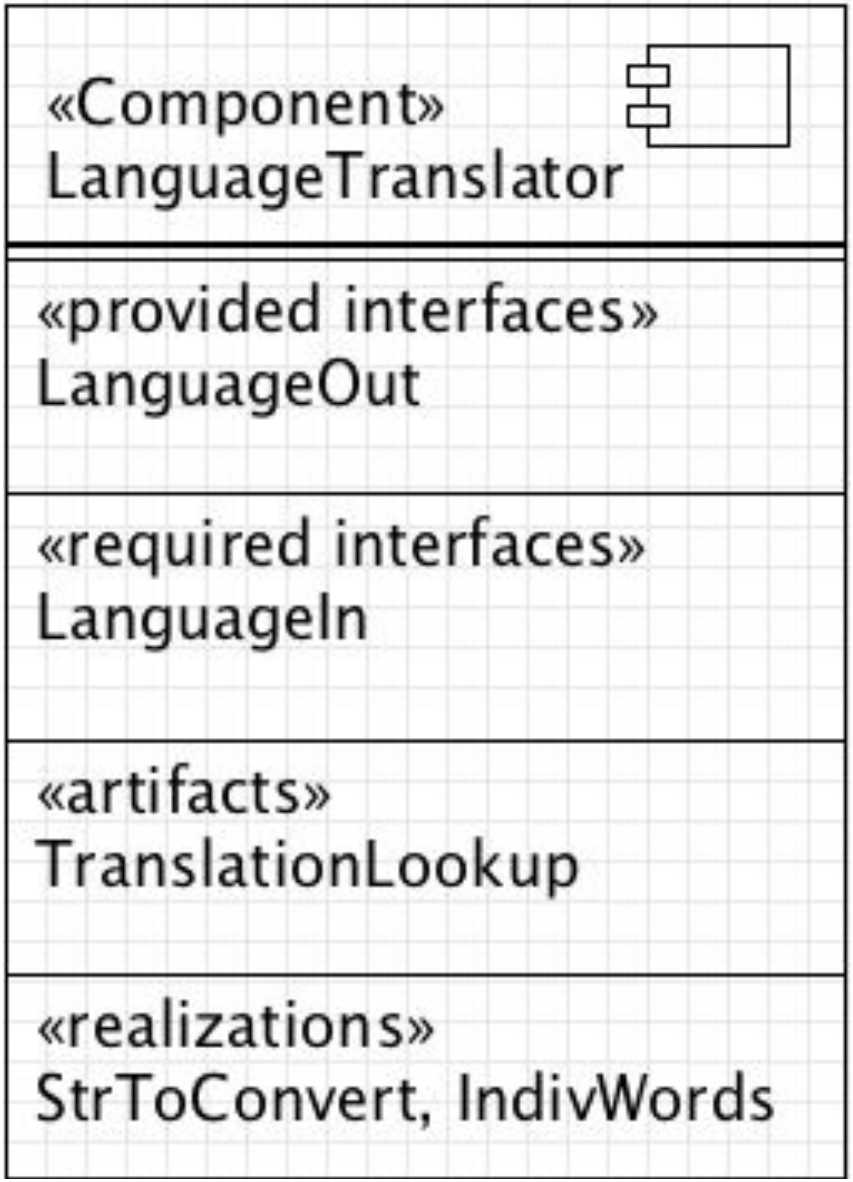
Connectors


«Component»
LangIn
«Component»
LanguageTrans
LangOut
«Component»
Assembly Connectors

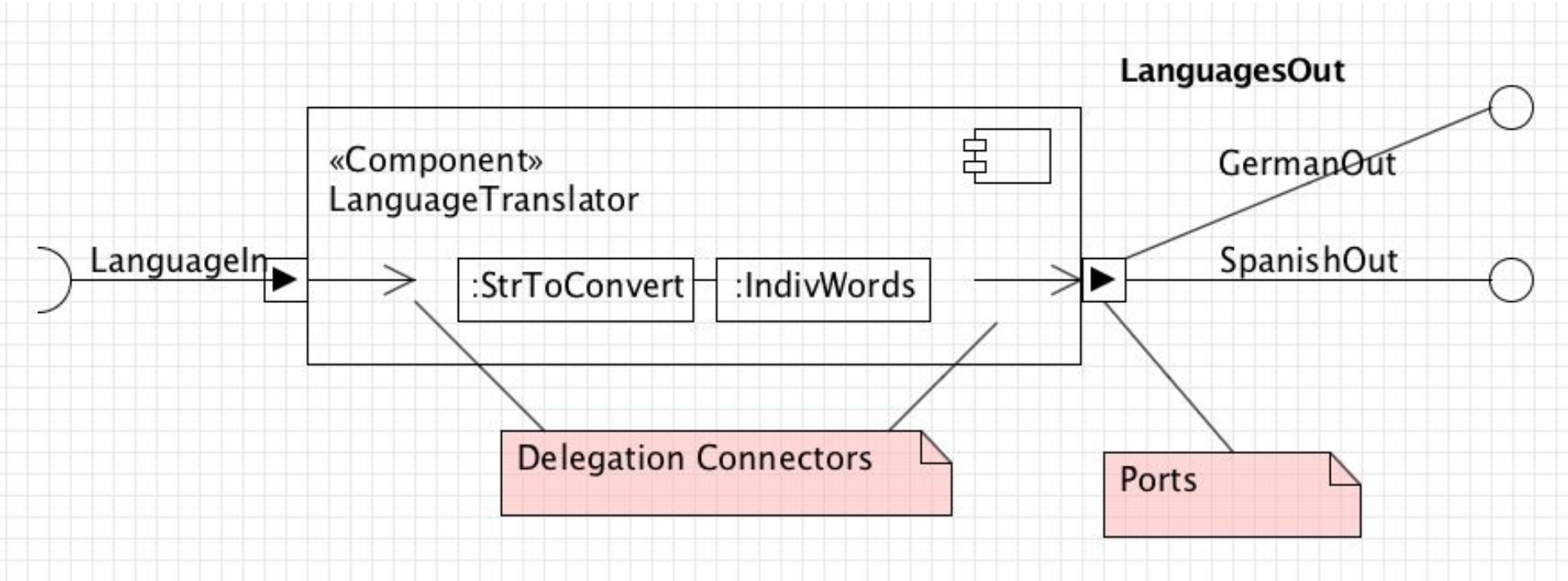
Realizations



Compact Alternative



Ports and Delegation Connectors



Summary

Review questions

Summary

- Ideas on Component Design and Architecture
- Design of existing application
  - Monolithic
  - Component-based
- UML
- Monolithic sys. are resistive to change
- Component-based sys. are adaptive to change

Review Questions

- What is tight coupling?
- Why is tight coupling not advisable?
- How do we measure the amount of coupling present in a design?
- What design strategies are used to lower coupling?
- Why should the business entity objects *not* be offered as dynamic component services?
- Why is coupling on a stable entity preferred over coupling on an unstable entity?
- Should componentization of multiple layer applications preserve the original layering structure? Why or why not?
- When changes are requested, is it advisable to change the provided and required interface definitions? Why or why not?